DATASTRUCTURES HOMEWORK-5

1) Use a linked list for polynomial representation and perform addition and subtraction of polynomials.

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff, exp;
    struct Node *next;
};

struct Node *create(int c, int e) {
    struct Node *n = malloc(sizeof(struct Node));
    n->coeff = c;
    n->exp = e;
    n->next = NULL;
    return n;
}

void insert(struct Node **head, int c, int e) {
    struct Node *n = create(c, e), *p;
    if (*head == NULL) {
        *head = n;
        return;
    }
    p = *head;
    while (p->next != NULL)
        p = p->next;
    p->next = n;
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next)
            printf(" + ");
        p = p->next;
    }
    printf("\n");
}

struct Node *operate(struct Node *a, struct Node *b, int sign) {
    struct Node *result = NULL;
```

```
    while (a && b) {
        if (a->exp == b->exp) {
            int c = a->coeff + sign * b->coeff;
            if (c != 0)
                insert(&result, c, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            insert(&result, a->coeff, a->exp);
            a = a->next;
        } else {
            insert(&result, sign * b->coeff, b->exp);
            b = b->next;
        }
    }

    while (a) {
        insert(&result, a->coeff, a->exp);
        a = a->next;
    }

    while (b) {
        insert(&result, sign * b->coeff, b->exp);
        b = b->next;
    }

    return result;
}

int main() {
    struct Node *p1 = NULL, *p2 = NULL, *r;

    insert(&p1, 3, 2);
    insert(&p1, 2, 1);
    insert(&p1, 1, 0);

    insert(&p2, 4, 2);
    insert(&p2, 5, 1);
    insert(&p2, 2, 0);

    printf("Polynomial 1: ");
    display(p1);

    printf("Polynomial 2: ");
    display(p2);
```

```
    r = operate(p1, p2, 1);
    printf("Addition: ");
    display(r);

    r = operate(p1, p2, -1);
    printf("Subtraction: ");
    display(r);

    return 0;
}
```

OUTPUT:

Polynomial 1: 3x^2 + 2x^1 + 1x^0

Polynomial 2: 4x^2 + 5x^1 + 2x^0

Addition: 7x^2 + 7x^1 + 3x^0

Subtraction: -1x^2 + -3x^1 + -1x^0

---

2) Find the previous node of a given element in a circular linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff, exp;
    struct Node *next;
};

struct Node *create(int c, int e) {
    struct Node *n = malloc(sizeof(struct Node));
    n->coeff = c;
    n->exp = e;
    n->next = NULL;
    return n;
}

void insert(struct Node **head, int c, int e) {
    struct Node *n = create(c, e), *p;
    if (*head == NULL) {
        *head = n;
        return;
    }
```

```
    p = *head;
    while (p->next != NULL)
        p = p->next;
    p->next = n;
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        if (p->next)
            printf(" + ");
        p = p->next;
    }
    printf("\n");
}

struct Node *operate(struct Node *a, struct Node *b, int sign) {
    struct Node *result = NULL;

    while (a && b) {
        if (a->exp == b->exp) {
            int c = a->coeff + sign * b->coeff;
            if (c != 0)
                insert(&result, c, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            insert(&result, a->coeff, a->exp);
            a = a->next;
        } else {
            insert(&result, sign * b->coeff, b->exp);
            b = b->next;
        }
    }

    while (a) {
        insert(&result, a->coeff, a->exp);
        a = a->next;
    }

    while (b) {
        insert(&result, sign * b->coeff, b->exp);
        b = b->next;
    }

    return result;
}
```

```c
int main() {
    struct Node *p1 = NULL, *p2 = NULL, *r;

    insert(&p1, 3, 2);
    insert(&p1, 2, 1);
    insert(&p1, 1, 0);

    insert(&p2, 4, 2);
    insert(&p2, 5, 1);
    insert(&p2, 2, 0);

    printf("Polynomial 1: ");
    display(p1);

    printf("Polynomial 2: ");
    display(p2);

    r = operate(p1, p2, 1);
    printf("Addition: ");
    display(r);

    r = operate(p1, p2, -1);
    printf("Subtraction: ");
    display(r);

    return 0;
}
```

OUTPUT:

Enter element: 30

Previous node: 20

---

3) Insert an element at a given position in a circular linked list.

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
```

```c
int main() {
    struct Node *head = NULL, *p, *n;
    int i, pos, value, count = 0;

    for (i = 1; i <= 3; i++) {
        n = malloc(sizeof(struct Node));
        n->data = i * 10;

        if (head == NULL) {
            head = n;
            n->next = head;
        } else {
            p = head;

            while (p->next != head)
                p = p->next;

            p->next = n;
            n->next = head;
        }
    }

    printf("Enter position and value: ");
    scanf("%d%d", &pos, &value);

    if (pos < 1) {
        printf("Invalid position\n");
        return 0;
    }

    n = malloc(sizeof(struct Node));
    n->data = value;

    if (pos == 1) {
        p = head;

        while (p->next != head)
            p = p->next;

        n->next = head;
        p->next = n;
        head = n;
    } else {
        p = head;
```

```
        while (count < pos - 2 && p->next != head) {
            p = p->next;
            count++;
        }

        if (count != pos - 2) {
            printf("Invalid position\n");
            return 0;
        }

        n->next = p->next;
        p->next = n;
    }

    printf("Circular linked list: ");

    p = head;

    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != head);

    printf("\n");

    return 0;
}
```

OUTPUT:
Enter position and value: 2 15

Circular linked list: 10 15 20 30

---

4) Find the second-largest distinct element in a queue.

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
    int q[100], n, i;
    int largest = INT_MIN, second = INT_MIN;

    printf("Enter queue size: ");
```

```
    scanf("%d", &n);

    if (n < 2 || n > 100) {
        printf("Invalid queue size\n");
        return 0;
    }

    printf("Enter elements: ");

    for (i = 0; i < n; i++)
        scanf("%d", &q[i]);

    for (i = 0; i < n; i++) {
        if (q[i] > largest) {
            second = largest;
            largest = q[i];
        } else if (q[i] > second && q[i] < largest) {
            second = q[i];
        }
    }

    if (second == INT_MIN)
        printf("No second-largest distinct element\n");
    else
        printf("Second largest element: %d\n", second);

    return 0;
}
```

OUTPUT:
Enter queue size: 5

Enter elements: 10 40 20 50 30

Second largest element: 40

---

5) Insert an element at the third position in a linked list and check whether the first and third elements are equal.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
```

```
    struct Node *next;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    return n;
}

void insertEnd(struct Node **head, int x) {
    struct Node *n = create(x), *p;

    if (*head == NULL) {
        *head = n;
        return;
    }

    p = *head;

    while (p->next != NULL)
        p = p->next;

    p->next = n;
}

int main() {
    struct Node *head = NULL, *p, *n;
    int i, value;

    for (i = 1; i <= 4; i++)
        insertEnd(&head, i * 10);

    printf("Enter value to insert at position 3: ");
    scanf("%d", &value);

    p = head;

    for (i = 1; i < 2; i++)
        p = p->next;

    n = create(value);
    n->next = p->next;
    p->next = n;

    printf("Linked list: ");
```

```
    p = head;

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }

    if (head->data == head->next->next->data)
        printf("\nFirst and third elements are equal\n");
    else
        printf("\nFirst and third elements are not equal\n");

    return 0;
}
```

OUTPUT:

Enter value to insert at position 3: 10

Linked list: 10 20 10 30 40

First and third elements are equal

---

6) Implement two stacks in one array and perform push, pop and display operations.

CODE:

```
#include <stdio.h>
#define SIZE 10

int a[SIZE];
int top1 = -1, top2 = SIZE;

void push1(int x) {
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
    else
        a[++top1] = x;
}

void push2(int x) {
    if (top1 + 1 == top2)
        printf("Stack Overflow\n");
    else
        a[--top2] = x;
}
```

```c
void pop1() {
    if (top1 == -1)
        printf("Stack 1 is empty\n");
    else
        printf("Popped: %d\n", a[top1--]);
}

void pop2() {
    if (top2 == SIZE)
        printf("Stack 2 is empty\n");
    else
        printf("Popped: %d\n", a[top2++]);
}

void display() {
    int i;

    printf("Stack 1: ");

    for (i = top1; i >= 0; i--)
        printf("%d ", a[i]);

    printf("\nStack 2: ");

    for (i = top2; i < SIZE; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main() {
    push1(10);
    push1(20);
    push2(90);
    push2(80);

    display();

    pop1();
    pop2();

    display();

    return 0;
}
```

OUTPUT:
Stack 1: 20 10

Stack 2: 80 90

Popped: 20

Popped: 80

Stack 1: 10

Stack 2: 90

---

7) Given a linked list containing n head nodes, where each node has a next pointer to the next head node and a bottom pointer to a sorted sub-linked list, flatten it into one sorted linked list.

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *bottom;
};

struct Node *create(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->next = NULL;
    n->bottom = NULL;
    return n;
}

struct Node *merge(struct Node *a, struct Node *b) {
    struct Node *result;

    if (a == NULL)
        return b;

    if (b == NULL)
        return a;

    if (a->data <= b->data) {
        result = a;
        result->bottom = merge(a->bottom, b);
    } else {
```

```
        result = b;
        result->bottom = merge(a, b->bottom);
    }

    result->next = NULL;

    return result;
}

struct Node *flatten(struct Node *head) {
    if (head == NULL || head->next == NULL)
        return head;

    head->next = flatten(head->next);

    return merge(head, head->next);
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->bottom;
    }

    printf("\n");
}

int main() {
    struct Node *head;

    head = create(5);
    head->next = create(10);
    head->next->next = create(19);
    head->next->next->next = create(28);

    head->bottom = create(7);
    head->bottom->bottom = create(8);
    head->bottom->bottom->bottom = create(30);

    head->next->bottom = create(20);

    head->next->next->bottom = create(22);
    head->next->next->bottom->bottom = create(50);

    head->next->next->next->bottom = create(35);
    head->next->next->next->bottom->bottom = create(40);
```

```
    head->next->next->next->bottom->bottom->bottom = create(45);

    head = flatten(head);

    printf("Flattened linked list:\n");
    display(head);

    return 0;
}
```

OUTPUT:

Flattened linked list:

5 7 8 10 19 20 22 28 30 35 40 45 50

---

8) Given K sorted linked lists of different sizes, merge them into one sorted linked list.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    return n;
}

void insert(struct Node **head, int x) {
    struct Node *n = create(x), *p;

    if (*head == NULL) {
        *head = n;
        return;
    }

    p = *head;

    while (p->next != NULL)
```

```
        p = p->next;

    p->next = n;
}

struct Node *merge(struct Node *a, struct Node *b) {
    struct Node dummy;
    struct Node *tail = &dummy;

    dummy.next = NULL;

    while (a != NULL && b != NULL) {
        if (a->data <= b->data) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }

        tail = tail->next;
    }

    tail->next = (a != NULL) ? a : b;

    return dummy.next;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }

    printf("\n");
}

int main() {
    struct Node *list[10] = {NULL};
    struct Node *result = NULL;
    int k, n, i, j, x;

    printf("Enter number of lists: ");
    scanf("%d", &k);

    if (k < 1 || k > 10) {
```

```
        printf("Invalid number of lists\n");
        return 0;
    }

    for (i = 0; i < k; i++) {
        printf("Enter size of list %d: ", i + 1);
        scanf("%d", &n);

        printf("Enter sorted elements: ");

        for (j = 0; j < n; j++) {
            scanf("%d", &x);
            insert(&list[i], x);
        }
    }

    for (i = 0; i < k; i++)
        result = merge(result, list[i]);

    printf("Merged sorted list: ");
    display(result);

    return 0;
}
```

OUTPUT:
Enter number of lists: 3

Enter size of list 1: 3

Enter sorted elements: 1 4 7

Enter size of list 2: 2

Enter sorted elements: 2 5

Enter size of list 3: 3

Enter sorted elements: 3 6 8

Merged sorted list: 1 2 3 4 5 6 7 8

---

9) Swap the Kth node from the beginning with the Kth node from the end in a singly linked list.

CODE:

```
#include <stdio.h>
```

```c
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    return n;
}

void insert(struct Node **head, int x) {
    struct Node *n = create(x), *p;

    if (*head == NULL) {
        *head = n;
        return;
    }

    p = *head;

    while (p->next != NULL)
        p = p->next;

    p->next = n;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }

    printf("\n");
}

void swapK(struct Node **head, int k) {
    struct Node dummy, *prevX, *prevY, *x, *y, *temp;
    int n = 0, i;

    dummy.next = *head;
    temp = *head;
```

```
    while (temp != NULL) {
        n++;
        temp = temp->next;
    }

    if (k < 1 || k > n) {
        printf("Invalid K\n");
        return;
    }

    if (2 * k - 1 == n)
        return;

    prevX = &dummy;

    for (i = 1; i < k; i++)
        prevX = prevX->next;

    prevY = &dummy;

    for (i = 1; i < n - k + 1; i++)
        prevY = prevY->next;

    x = prevX->next;
    y = prevY->next;

    if (x->next == y) {
        prevX->next = y;
        x->next = y->next;
        y->next = x;
    } else if (y->next == x) {
        prevY->next = x;
        y->next = x->next;
        x->next = y;
    } else {
        temp = x->next;
        x->next = y->next;
        y->next = temp;

        prevX->next = y;
        prevY->next = x;
    }

    *head = dummy.next;
}
```

```c
int main() {
    struct Node *head = NULL;
    int n, i, x, k;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");

    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        insert(&head, x);
    }

    printf("Enter K: ");
    scanf("%d", &k);

    swapK(&head, k);

    printf("After swapping: ");
    display(head);

    return 0;
}
```

OUTPUT:

Enter number of nodes: 5

Enter elements: 10 20 30 40 50

Enter K: 2

After swapping: 10 40 30 20 50

---

10) Multiply two numbers represented by linked lists, where each node contains one digit.

CODE:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
```

```c
struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    return n;
}

void insert(struct Node **head, int x) {
    struct Node *n = create(x), *p;

    if (*head == NULL) {
        *head = n;
        return;
    }

    p = *head;

    while (p->next != NULL)
        p = p->next;

    p->next = n;
}

long long getNumber(struct Node *head) {
    long long num = 0;

    while (head != NULL) {
        num = num * 10 + head->data;
        head = head->next;
    }

    return num;
}

void readList(struct Node **head) {
    int n, i, digit;

    printf("Enter number of digits: ");
    scanf("%d", &n);

    printf("Enter digits: ");

    for (i = 0; i < n; i++) {
        scanf("%d", &digit);
        insert(head, digit);
    }
```

```c
}

int main() {
    struct Node *a = NULL, *b = NULL;
    long long num1, num2, product;

    printf("First number:\n");
    readList(&a);

    printf("Second number:\n");
    readList(&b);

    num1 = getNumber(a);
    num2 = getNumber(b);

    product = num1 * num2;

    printf("First number: %lld\n", num1);
    printf("Second number: %lld\n", num2);
    printf("Product: %lld\n", product);

    return 0;
}
```

OUTPUT:

First number:

Enter number of digits: 3

Enter digits: 1 0 0

Second number:

Enter number of digits: 2

Enter digits: 1 0

First number: 100

Second number: 10

Product: 1000
